



UNIVERSIDAD INTERNACIONAL DEL ECUADOR

Powered by Arizona State University

FACULTAD DE SISTEMAS DE INFORMACIÓN

Carrera de Ingeniería en Ciberseguridad

APRENDIZAJE AUTÓNOMO 1

BASTION: diseño funcional y arquitectónico de un gestor personal de credenciales con generador de contraseñas seguras

Estudiante: David Alexander Benz Zambrano

Correo institucional: dabenzza@uide.edu.ec

Carrera: Ingeniería en Ciberseguridad

Semestre: Primero · **Paralelo:** 1-CIB-1A

Asignatura: Lógica de Programación

Docente: Msc. Lilian Aman

Quito - Ecuador · mayo de 2026

ÍNDICE DE CONTENIDOS

1. INTRODUCCIÓN	7
1.1 Contexto y problema	7
1.2 Contexto profesional: Blue Nova Consulting	7
1.3 Objetivos	7
1.4 Metodología y uso de inteligencia artificial	8
2. EL PROBLEMA Y EL CASO SELECCIONADO	8
2.1 Historias de usuario	8
2.2 Variables y tipos de datos	8
2.3 Entradas, salidas y reglas de validación	9
3. LA SOLUCIÓN: BASTION	9
3.1 Qué es y propuesta de valor	9
3.2 Características y alcance del MVP	10
3.3 Comparación con el mercado	10
4. ENFOQUE DE DISEÑO E INVESTIGACIÓN	12
5. DISEÑO FUNCIONAL	12
5.1 Tipos de diagramas investigados y selección	12
5.2 Diagrama de casos de uso	13
5.3 Diagrama de flujo	15
6. DISEÑO DE LA ARQUITECTURA	17
6.1 Estilos de arquitectura investigados y selección	17
6.2 Arquitectura en capas	17
6.3 Stack tecnológico	18
6.4 Implementación en Python	19
7. SEGURIDAD Y ESTÁNDARES	19
7.1 Estándares aplicados	19
7.2 Modelo de cifrado y amenazas	20
7.3 Tendencias actuales (2026)	21

8. PROTOTIPO DE INTERFAZ	21
9. ALCANCE DEL PERIODO Y PROYECCIÓN	22
9.1 Alcance del periodo	22
9.2 Aplicación de los módulos de la asignatura	23

ÍNDICE DE TABLAS

Tabla 1. Historias de usuario de BASTION	8
Tabla 2. Variables del proyecto clasificadas por tipo de dato	9
Tabla 3. Reglas de validación de entrada	9
Tabla 4. Comparativa de gestores de contraseñas frente a BASTION	10
Tabla 5. Tipos investigados, selección y dónde se aplican en BASTION	12
Tabla 6. Tipos de diagramas de funcionalidad: ejemplo y uso en BASTION	13
Tabla 7. Especificación del caso de uso CU-01: Crear bóveda	14
Tabla 8. Especificación del caso de uso CU-03: Generar contraseña	15
Tabla 9. Estilos de arquitectura de software: ejemplo y uso en BASTION	17
Tabla 10. Stack tecnológico del MVP y su justificación	18
Tabla 11. Estándares y normas aplicadas en el diseño de BASTION	20

ÍNDICE DE FIGURAS

Figura 1. Diagrama de casos de uso del sistema BASTION (UML).	14
Figura 2. Diagrama de flujo del proceso principal de BASTION (ISO 5807).	16
Figura 3. Arquitectura en capas de BASTION.	18
Figura 4. Modelo de cifrado de credenciales en BASTION.	20
Figura 5. Prototipo de alta fidelidad: pantalla principal de la bóveda de BASTION.	22
Figura 6. Alcance del periodo (MVP en junio) y proyección de BASTION.	23

LISTA DE ACRÓNIMOS Y SIGLAS

2FA	Autenticación de Dos Factores (Two-Factor Authentication)
AES	Advanced Encryption Standard
API	Application Programming Interface
APA	American Psychological Association
CLI	Command-Line Interface (interfaz de línea de comandos)
GCM	Galois/Counter Mode
GUI	Graphical User Interface (interfaz gráfica de usuario)
IA	Inteligencia Artificial
IEEE	Institute of Electrical and Electronics Engineers
IETF	Internet Engineering Task Force
ISO	International Organization for Standardization
KDF	Key Derivation Function (función de derivación de clave)
LOPD	Ley Orgánica de Protección de Datos Personales (Ecuador)
MVP	Minimum Viable Product (producto mínimo viable)
NIST	National Institute of Standards and Technology
OMG	Object Management Group
OTP	One-Time Password (contraseña de un solo uso)
PMI	Project Management Institute
RFC	Request For Comments
SSH	Secure Shell
TOTP	Time-based One-Time Password
UML	Unified Modeling Language
UIDE	Universidad Internacional del Ecuador

RESUMEN

El presente trabajo desarrolla el diseño funcional y arquitectónico de BASTION, un gestor personal de credenciales que integra un generador de contraseñas seguras con una bóveda local cifrada, concebido como solución al problema base “generador seguro de contraseñas” propuesto en la asignatura. El objetivo de esta primera fase no es programar, sino comprender el problema y modelar la solución antes de escribir código, aplicando diagramas de funcionalidad y de arquitectura de software conforme a estándares reconocidos por la industria.

La metodología combina la ingeniería de requisitos descrita por Sommerville (2016) y Pressman y Maxim (2021) con el modelado en UML normalizado por el Object Management Group (2017) y los estándares criptográficos del NIST y la IETF. Se elaboran un diagrama de casos de uso, un diagrama de flujo del proceso principal, un diagrama de arquitectura en capas y un diagrama del modelo de cifrado; adicionalmente se construye un prototipo funcional de interfaz de alta fidelidad que materializa el diseño propuesto. El producto se contextualiza en la nueva línea de ciberseguridad de Blue Nova Consulting, agencia ecuatoriana de la que el estudiante es socio fundador. En el proceso se empleó inteligencia artificial generativa como herramienta para optimizar tareas y acelerar los tiempos de desarrollo, manteniéndose en el autor las decisiones de diseño, el alcance y la validación, según se detalla en la sección 1.5.

Los resultados muestran que un gestor de credenciales bien diseñado se sostiene sobre tres pilares: una generación de contraseñas con aleatoriedad criptográfica (RFC 4086), un cifrado autenticado moderno (AES-256-GCM con derivación de clave Argon2id) y una arquitectura modular en capas que separa la interfaz, la lógica de negocio y la persistencia. El diseño resultante es comparable en modelo de seguridad con soluciones del mercado como Bitwarden o KeePass, manteniendo un alcance realista para un MVP desarrollable en Python durante el periodo académico.

Palabras clave: gestor de contraseñas, generador seguro, cifrado AES-256-GCM, diagramas UML, arquitectura de software, ciberseguridad.

PARTE I

BASTION: el problema y la solución

Qué problema se resuelve, por qué se eligió este caso y en qué consiste el producto.

1. INTRODUCCIÓN

1.1 Contexto y problema

La gestión de contraseñas es uno de los problemas de seguridad más cotidianos y peor resueltos por el usuario promedio. La proliferación de servicios digitales multiplica las credenciales que una persona administra, lo que deriva en hábitos inseguros: reutilizar la misma contraseña, usar claves cortas y predecibles o guardarlas en archivos de texto sin protección. Las guías de identidad digital del NIST advierten que la fortaleza de una credencial y su unicidad por servicio son determinantes para resistir ataques de fuerza bruta y de relleno de credenciales (Grassi et al., 2017).

Los gestores de contraseñas son la herramienta recomendada por la literatura de seguridad: generan claves robustas, las almacenan cifradas y las recuperan mediante una única contraseña maestra (Stallings, 2023). BASTION se inscribe en esta categoría: parte de la opción «generador seguro de contraseñas» del enunciado y la extiende hacia una solución que no solo crea contraseñas, sino que las custodia en una bóveda local cifrada.

1.2 Contexto profesional: Blue Nova Consulting

El proyecto tiene un doble propósito. Como producto académico, cumple los objetivos de la actividad de diseño previo a la codificación. Como herramienta real, responde a una necesidad de Blue Nova Consulting, agencia ecuatoriana de soluciones digitales de la que el autor es socio fundador y que abre una línea de ciberseguridad. El equipo administra a diario credenciales sensibles (accesos a servidores, claves de API y llaves SSH) hoy dispersas. BASTION nace para centralizarlas y protegerlas bajo una sola bóveda cifrada, primero como uso interno y, después, como base de un servicio para clientes.

1.3 Objetivos

Objetivo general. Diseñar, en su fase previa a la codificación, la solución funcional y arquitectónica de BASTION mediante diagramas de funcionalidad y de arquitectura fundamentados en estándares de la industria.

Objetivos específicos:

- Investigar y seleccionar los tipos de diagramas de funcionalidad y de arquitectura pertinentes para modelar el sistema.
- Analizar el problema de la gestión de contraseñas e identificar funcionalidades, variables, datos de entrada y salida y reglas de validación.
- Diseñar la arquitectura en capas y el modelo de seguridad, justificando el stack y los estándares criptográficos aplicados.
- Construir un prototipo funcional de alta fidelidad que valide el diseño y sirva de guía para la implementación en Python.

1.4 Metodología y uso de inteligencia artificial

El trabajo sigue un enfoque de ingeniería de software centrado en el diseño previo: análisis del problema y elicitación de requisitos (Sommerville, 2016), modelado en UML (Object Management Group, 2017) y diseño de la arquitectura en capas (Pressman y Maxim, 2021). Las decisiones de seguridad se fundamentan en estándares del NIST y la IETF, y todas las fuentes se citan bajo las normas APA en su séptima edición (American Psychological Association, 2020).

En la elaboración se empleó inteligencia artificial generativa como herramienta para optimizar tareas y acelerar los tiempos de desarrollo (investigación de fuentes, organización de la documentación y prototipado), de forma comparable a un entorno de desarrollo. Las decisiones de diseño, el alcance y la validación de cada resultado corresponden al criterio del autor; la IA no sustituye el juicio profesional.

2. EL PROBLEMA Y EL CASO SELECCIONADO

El enunciado de la actividad ofrece varias opciones de programa; de ellas se tomó el «generador seguro de contraseñas» por ser la más alineada con una necesidad real de ciberseguridad. El problema central es la administración insegura de múltiples credenciales por un mismo usuario, que se manifiesta en la reutilización de contraseñas, el uso de claves débiles y su almacenamiento sin cifrar. La solución debe generar contraseñas fuertes y únicas, custodiarlas cifradas y recuperarlas con comodidad mediante una contraseña maestra.

2.1 Historias de usuario

Tabla 1
Historias de usuario de BASTION

ID	Como usuario quiero...	Para...
HU-01	generar una contraseña fuerte con un clic	no tener que inventarla yo mismo.
HU-02	guardar mis credenciales en un solo lugar cifrado	dejar de usar archivos de texto inseguros.
HU-03	abrir todo con una sola contraseña maestra	no memorizar decenas de claves.
HU-04	organizar mis credenciales por categorías	encontrarlas con orden.
HU-05	buscar una credencial por su nombre	acceder a ella rápidamente.
HU-06	ver la fortaleza de cada contraseña	saber cuáles debo reemplazar.

2.2 Variables y tipos de datos

El análisis de los datos del programa permite identificar las variables principales y su tipo, aplicando los tipos básicos de la asignatura. Como muestra la Tabla 2, el sistema emplea cadenas, enteros y booleanos; el tipo decimal no se utiliza porque ninguna magnitud del dominio requiere fracciones.

Tabla 2

Variables del proyecto clasificadas por tipo de dato

Variable	Tipo	Ejemplo	Descripción
contrasena_maestra	str	“MiClave2026!”	Clave que descifra la bóveda.
longitud_deseada	int	16	Largo de la contraseña a generar.
incluir_simbolos	bool	True	Incluir símbolos en la generación.
incluir_numeros	bool	True	Incluir dígitos.
categoria	str	“Redes sociales”	Grupo de la credencial.
nombre_servicio	str	“Instagram”	Servicio asociado.
contrasena_generada	str	“K9\$mP2!xQ”	Salida del generador.

Nota. Aplica tres de los cuatro tipos básicos vistos en clase: str, int y bool.

2.3 Entradas, salidas y reglas de validación

Los **datos de entrada** son la contraseña maestra, los parámetros del generador (longitud y tipos de caracteres) y los datos de cada credencial (servicio, usuario, categoría). Los **datos de salida** son la contraseña generada, la bóveda cifrada actualizada, los resultados de búsqueda y el indicador de fortaleza. La Tabla 3 resume las reglas de validación de cada entrada.

Tabla 3

Reglas de validación de entrada

Entrada	Regla de validación
Contraseña maestra	Longitud mínima recomendada; no puede quedar vacía al crear la bóveda.
Longitud de contraseña	Número entero dentro de un rango permitido (por ejemplo, 6 a 40).
Tipos de caracteres	Debe seleccionarse al menos un conjunto de caracteres.
Nombre de servicio	Campo obligatorio y no vacío al guardar una credencial.
Categoría	Debe corresponder a una de las categorías predefinidas.

3. LA SOLUCIÓN: BASTION

3.1 Qué es y propuesta de valor

BASTION es una bóveda personal de credenciales: una «caja fuerte» en el computador del usuario que genera contraseñas seguras y las guarda en un archivo cifrado, protegido por una única contraseña maestra. Su lema de diseño es simple: una sola clave para custodiar todas las demás. Su valor reside en combinar seguridad (cifrado autenticado moderno y ausencia de nube, que reduce la superficie de ataque) con simplicidad: una interfaz centrada en cuatro acciones esenciales (generar, guardar, buscar y ver).

3.2 Características y alcance del MVP

- **Generación de contraseñas** con longitud y composición configurables, basada en aleatoriedad criptográfica.
- **Bóveda cifrada local:** todas las credenciales en un único archivo cifrado, sin enviarse a ningún servidor.
- **Contraseña maestra** única que descifra la bóveda; sin ella el archivo es ilegible.
- **Organización por categorías y búsqueda** rápida por servicio o usuario.
- **Indicador de fortaleza** que clasifica cada contraseña como débil, media o fuerte.

Estas funciones constituyen el MVP entregable durante el periodo. Quedan deliberadamente fuera la sincronización en la nube, el uso multi-dispositivo, el autocompletado en navegadores y el doble factor TOTP, que se planifican para versiones posteriores (capítulo 9). Delimitar el alcance asegura una primera versión sólida y terminada.

3.3 Comparación con el mercado

Se revisaron cuatro gestores ampliamente adoptados para delimitar el espacio de BASTION. Bitwarden es de código abierto y basado en la nube; 1Password es comercial y orientado a equipos; LastPass popularizó el modelo en la nube; y KeePass es un gestor local de código abierto que guarda las credenciales en un archivo cifrado en el propio equipo (Stallings, 2023). BASTION se aproxima a KeePass por su enfoque local y la ausencia de nube en su MVP, con una especialización en perfiles técnicos que gestionan también claves de API y llaves SSH. La Tabla 4 resume la comparación.

Tabla 4

Comparativa de gestores de contraseñas frente a BASTION

Característica	Bitwarden	1Password	LastPass	KeePass	BASTION (MVP)
Almacenamiento	Nube	Nube	Nube	Local	Local
Código abierto	Sí	No	No	Sí	Sí
Cifrado	AES-256	AES-256	AES-256	AES-256	AES-256-GCM
Doble factor	Sí	Sí	Sí	Plugin	TOTP (futuro)
Costo	Freemium	Pago	Freemium	Gratuito	Gratuito
Enfoque	General	Equipos	General	Técnico	Técnico / interno

Nota. Elaboración propia con base en Stallings (2023) y la documentación pública de cada producto.

PARTE II

Diseño de la solución

La investigación de diagramas aplicada a cada decisión de diseño, con su fuente.

4. ENFOQUE DE DISEÑO E INVESTIGACIÓN

Coherente con el principio de diseñar antes de programar, cada decisión de diseño parte de una investigación de los tipos de diagramas disponibles, de la que se selecciona el más adecuado para BASTION. La Tabla 5 resume, para cada aspecto, qué tipos se investigaron, cuál se tomó, en qué parte del documento se aplica y la fuente que lo respalda. Los apartados siguientes desarrollan cada selección.

Tabla 5

Tipos investigados, selección y dónde se aplican en BASTION

Aspecto	Tipos investigados	Seleccionado	Se aplica en	Fuente
Funcionalidad	Casos de uso, actividad, secuencia, diagrama de flujo	Casos de uso y diagrama de flujo	Cap. 5 (Figuras 1 y 2)	OMG (2017); Booch et al. (2005)
Arquitectura	Monolítico, capas, MVC, cliente-servidor, microservicios	Arquitectura en 3 capas	Cap. 6 (Figura 3)	Bass et al. (2012); Sommerville (2016)

Nota. Elaboración propia. La selección se justifica en los capítulos 5 y 6.

5. DISEÑO FUNCIONAL

5.1 Tipos de diagramas investigados y selección

Para modelar qué hace el sistema antes de programarlo se investigaron los principales diagramas de funcionalidad. El lenguaje UML reúne los más usados, como los de casos de uso, actividad y secuencia (Booch et al., 2005; Object Management Group, 2017), y, anterior a UML, el diagrama de flujo, normalizado en la norma ISO 5807. El de casos de uso modela funciones y actores; el de actividad detalla flujos con concurrencia; el de secuencia muestra mensajes entre objetos; y el de flujo representa la secuencia de pasos y decisiones de un proceso (Pressman y Maxim, 2021).

Se seleccionan **dos**: el **diagrama de casos de uso**, que captura qué hace BASTION y quién lo usa, y el **diagrama de flujo**, que describe la lógica del proceso principal. Los diagramas de actividad y de secuencia se descartan en esta etapa porque introducen un nivel de detalle de implementación que excede un diseño previo a la programación.

La Tabla 6 resume los principales tipos de diagramas de funcionalidad: qué modela cada uno, un ejemplo típico y cuáles se aplican en BASTION. Los dos seleccionados se desarrollan, con el ejemplo real del sistema, en los apartados siguientes.

Tabla 6

Tipos de diagramas de funcionalidad: ejemplo y uso en BASTION

Tipo	Qué modela	Ejemplo típico	¿En BASTION?
Casos de uso	Las funciones del sistema y los actores que las ejecutan	Un usuario que «inicia sesión» o «genera una contraseña»	Sí (Figura 1)
Diagrama de flujo	La secuencia de pasos y decisiones de un proceso	Validar la contraseña maestra y abrir la bóveda paso a paso	Sí (Figura 2)
Actividad	Flujos de trabajo con acciones en paralelo y bifurcaciones	Un proceso de compra con tareas que ocurren a la vez	No (innecesario en esta fase)
Secuencia	Los mensajes entre objetos a lo largo del tiempo	El intercambio entre interfaz, lógica y almacenamiento	No (nivel de implementación)

Nota. Elaboración propia con base en Booch et al. (2005) y Object Management Group (2017).

5.2 Diagrama de casos de uso

Se identificaron siete casos de uso iniciados por un único actor, el Usuario. La Figura 1 los representa en notación UML; las relaciones «include» indican que agregar una credencial reutiliza la generación de contraseñas y que ver la bóveda organizada reutiliza la búsqueda. A continuación se especifican los dos casos de uso más representativos.

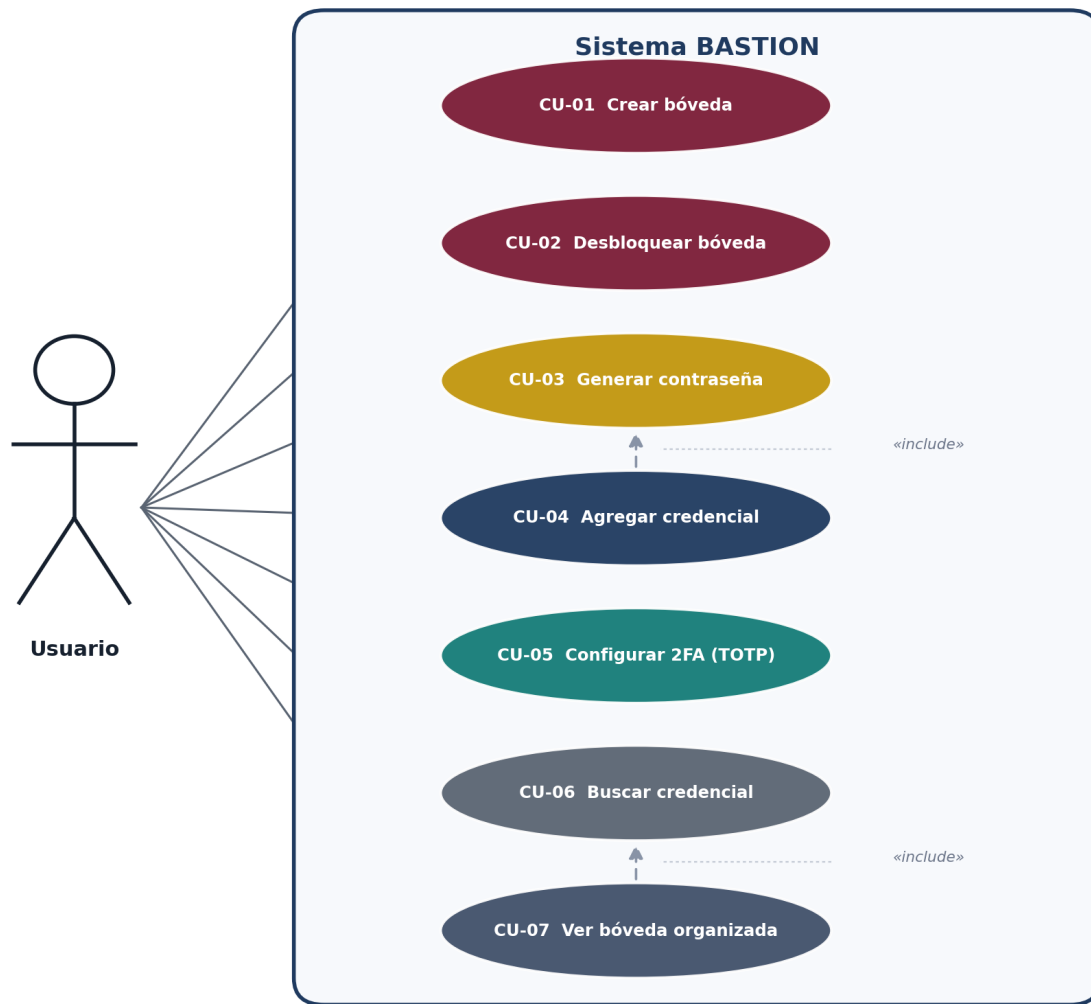


Figura 1. Diagrama de casos de uso del sistema BASTION (UML).

Tabla 7

Especificación del caso de uso CU-01: Crear bóveda

Campo	Descripción
Actor	Usuario.
Precondición	No existe una bóveda previa en el equipo.
Flujo principal	1) El usuario abre BASTION. 2) El sistema detecta que no hay bóveda. 3) Solicita definir una contraseña maestra. 4) Cifra y guarda un archivo de bóveda vacío. 5) Abre el menú principal.
Postcondición	Existe una bóveda cifrada lista para almacenar credenciales.

Tabla 8

Especificación del caso de uso CU-03: Generar contraseña

Campo	Descripción
Actor	Usuario.
Precondición	La bóveda está abierta (sesión iniciada).
Flujo principal	1) El usuario indica longitud y tipos de caracteres. 2) El sistema valida los parámetros. 3) Genera la contraseña con aleatoriedad criptográfica. 4) Calcula y muestra su fortaleza. 5) Permite copiarla o usarla en una credencial.
Postcondición	Se obtiene una contraseña segura lista para usarse o guardarse.

5.3 Diagrama de flujo

La Figura 2 modela el flujo del proceso principal: desde la apertura del programa y la verificación de la existencia de la bóveda, pasando por la autenticación con la contraseña maestra, hasta el ciclo de acciones del usuario y el guardado cifrado de los cambios.

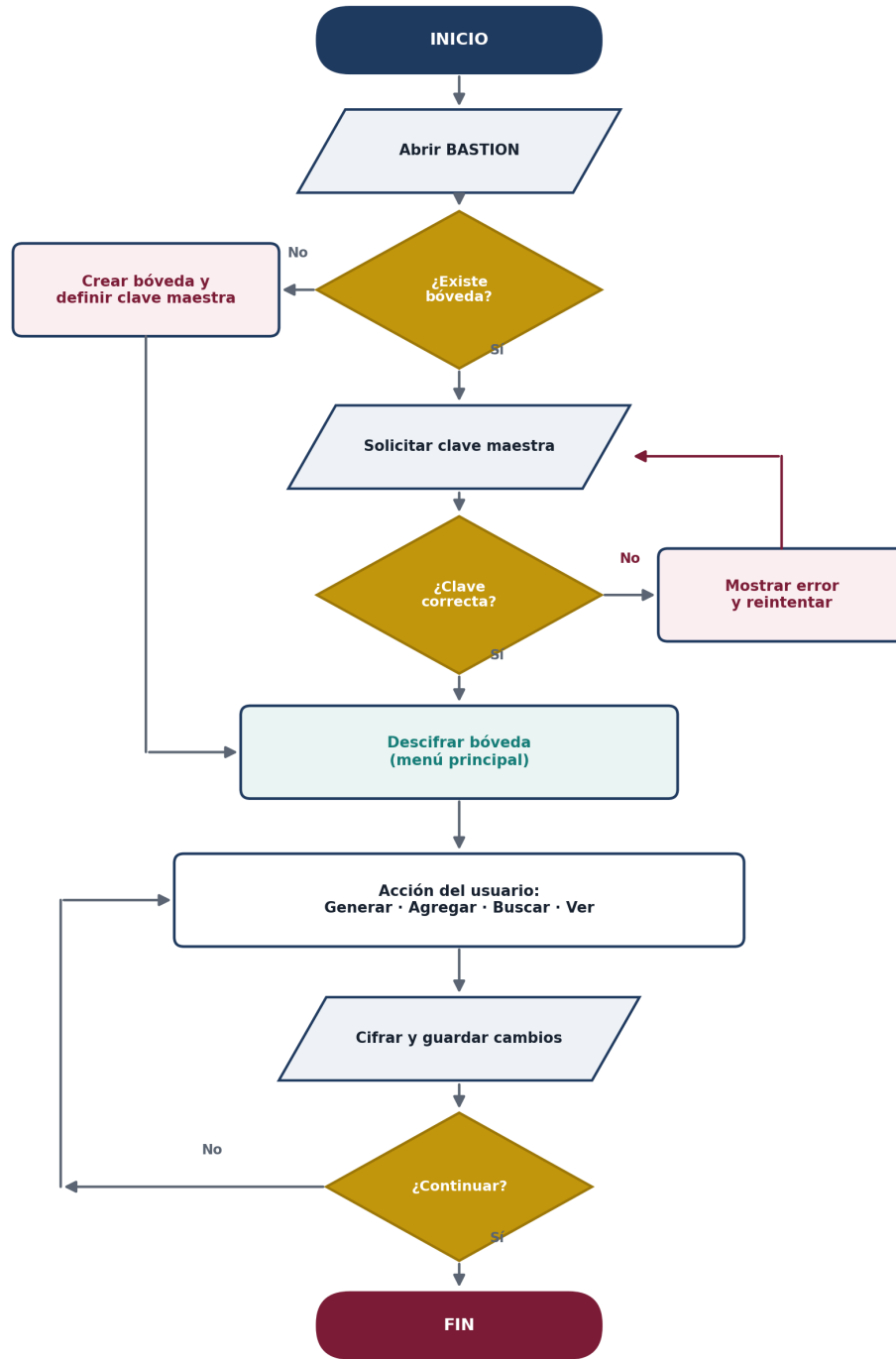


Figura 2. Diagrama de flujo del proceso principal de BASTION (ISO 5807).

6. DISEÑO DE LA ARQUITECTURA

6.1 Estilos de arquitectura investigados y selección

La arquitectura de software es la organización fundamental de un sistema: sus componentes, sus relaciones y los principios que guían su diseño (Bass et al., 2012; Sommerville, 2016). Se investigaron varios estilos: el monolítico, el de capas, el modelo-vista-controlador (MVC), el cliente-servidor y los microservicios (Fowler, 2002; Pressman y Maxim, 2021). El patrón en capas organiza el sistema en niveles jerárquicos; MVC separa datos, presentación y control; el cliente-servidor reparte el sistema entre servidor y clientes; y los microservicios lo descomponen en servicios independientes.

Para BASTION se selecciona la **arquitectura en tres capas**. Frente a MVC o cliente-servidor, refleja con mayor claridad la separación entre la interfaz, las reglas del dominio y el almacenamiento cifrado; los microservicios se descartan por innecesarios en una aplicación local de escritorio. El estilo elegido permite evolucionar una capa sin afectar a las demás (Sommerville, 2016).

La Tabla 9 resume los estilos de arquitectura considerados: la idea central de cada uno, un ejemplo típico y cuál se aplica en BASTION. El estilo seleccionado se desarrolla, con el diagrama real del sistema, en el apartado siguiente.

Tabla 9

Estilos de arquitectura de software: ejemplo y uso en BASTION

Estilo	Idea central	Ejemplo típico	¿En BASTION?
En capas	Niveles jerárquicos: presentación, lógica y datos	App de escritorio con interfaz, reglas y archivo local	Sí (Figura 3)
Monolítico	Todo el código en una sola unidad, sin separar	Un único script sin división de responsabilidades	No (poco mantenible)
MVC	Separa modelo, vista y controlador	Una aplicación web con plantillas y controladores	No
Cliente-servidor	Un servidor central atiende a varios clientes	Un gestor en la nube consultado por varias apps	No (BASTION es local)
Microservicios	Servicios independientes que se comunican entre sí	Una plataforma con servicios desplegables por separado	No (excesivo aquí)

Nota. Elaboración propia con base en Bass et al. (2012), Fowler (2002) y Sommerville (2016).

6.2 Arquitectura en capas

BASTION se estructura en tres capas, como ilustra la Figura 3. La **capa de presentación** contiene la interfaz (menús, formularios e indicador de fortaleza). La **capa de lógica de negocio** concentra el generador, el motor de cifrado, el gestor de credenciales, las validaciones, el buscador y, a futuro, el doble factor. La **capa de persistencia** lee y escribe el archivo local cifrado. Esta separación permite sustituir una interfaz de línea de comandos por una gráfica sin alterar la lógica ni el almacenamiento (Pressman y Maxim, 2021).



Figura 3. Arquitectura en capas de BASTION.

6.3 Stack tecnológico

La Tabla 10 detalla las tecnologías del MVP y la razón de cada elección. La decisión central es el uso de Python por su legibilidad, su ecosistema de bibliotecas criptográficas maduras y su idoneidad para un primer semestre.

Tabla 10
Stack tecnológico del MVP y su justificación

Componente	Tecnología	Justificación
Lenguaje	Python 3.13+	Legible, multiplataforma y con ecosistema criptográfico maduro.
Cifrado	AES-256-GCM	Cifrado autenticado estándar del NIST (SP 800-38D).
Derivación de clave	Argon2id	Recomendación de OWASP (2025) para contraseñas; RFC 9106.
Aleatoriedad	secrets / os.urandom	Aleatoriedad criptográfica del sistema operativo.
Persistencia	Archivo local cifrado	Sin nube; reduce la superficie de ataque.
Interfaz	CLI / GUI	CLI para el MVP; interfaz gráfica como evolución natural.

Nota. Elaboración propia con base en Dworkin (2007), Barker (2020) y la documentación de Python.

6.4 Implementación en Python

Un principio rector es no programar la criptografía desde cero, sino apoyarse en módulos especializados; el código propio se concentra en la lógica del dominio. Las bibliotecas previstas son:

- **secrets**: genera las contraseñas con aleatoriedad criptográfica (no usa el módulo `random` común).
- **string**: provee los conjuntos de caracteres: letras, dígitos y símbolos.
- **cryptography**: cifra y descifra la bóveda con AES-256-GCM.
- **argon2-cffi**: deriva la clave de cifrado a partir de la contraseña maestra (Argon2id).
- **json**: estructura y serializa las credenciales antes de cifrarlas.
- **getpass**: lee la contraseña maestra por teclado sin mostrarla.

El componente central y primero en construirse es el generador. El siguiente fragmento muestra una versión inicial, alcanzable con los contenidos de la asignatura:

```
import secrets
import string

def generar_contrasena(longitud, usar_simbolos):
    alfabeto = string.ascii_letters + string.digits
    if usar_simbolos:
        alfabeto += "!@#$%&*?-_ "
    clave = ""
    for i in range(longitud):
        clave += secrets.choice(alfabeto)
    return clave

contrasena = generar_contrasena(16, True)
print("Contrasena segura:", contrasena)
```

Este fragmento evidencia la transferencia directa de los contenidos de Lógica de Programación: **variables**, una **estructura de decisión** (`if`), un **bucle** (`for`) y el encapsulamiento en una **función**. La diferencia de seguridad está en usar `secrets.choice` en lugar de un generador pseudoaleatorio común, conforme a la RFC 4086.

7. SEGURIDAD Y ESTÁNDARES

7.1 Estándares aplicados

Un gestor de contraseñas es, ante todo, un sistema criptográfico, por lo que su diseño se apoya en estándares probados en lugar de mecanismos ad hoc. La Tabla 11 resume los estándares que sustentan a BASTION.

Tabla 11
Estándares y normas aplicadas en el diseño de BASTION

Ámbito	Estándar / norma	Aporte al diseño
Aleatoriedad	RFC 4086 (IETF)	Aleatoriedad criptográfica para contraseñas impredecibles.
Cifrado	AES-256-GCM · NIST SP 800-38D	Cifrado autenticado: confidencialidad e integridad.
Derivación de clave	Argon2id · RFC 9106	Convierte la clave maestra en clave resistente a hardware.
Doble factor	TOTP · RFC 6238	Contraseñas temporales de un solo uso (versión futura).
Identidad	NIST SP 800-63B	Fortaleza y unicidad de credenciales.
Modelado	UML 2.5.1 · OMG	Notación de los diagramas de casos de uso.

Nota. Elaboración propia con base en Eastlake et al. (2005), Dworkin (2007), M'Raihi et al. (2011), Grassi et al. (2017), Biryukov et al. (2021) y Object Management Group (2017).

7.2 Modelo de cifrado y amenazas

El principal activo a proteger es la bóveda. Las amenazas consideradas son el robo del archivo, el acceso no autorizado al equipo y los ataques de fuerza bruta contra la contraseña maestra. La Figura 4 representa el flujo criptográfico: la contraseña maestra no se usa directamente como clave, sino que Argon2id la transforma en una clave robusta; con ella, AES-256-GCM cifra cada credencial y produce el archivo de bóveda. Sin la contraseña correcta no es posible derivar la clave y el archivo permanece ilegible (Barker, 2020; Dworkin, 2007).



Figura 4. *Modelo de cifrado de credenciales en BASTION.*

7.3 Tendencias actuales (2026)

El diseño de un gestor de credenciales en 2026 considera dos transformaciones recientes. La primera son los **passkeys**: credenciales FIDO2/WebAuthn que sustituyen la contraseña por un par de claves criptográficas y son resistentes al phishing, pues el servidor solo guarda la clave pública (FIDO Alliance, 2025); la guía NIST SP 800-63-4 (2025) los reconoce como autenticadores de nivel AAL2. La segunda es la **transición post-cuántica**: en 2024 el NIST publicó FIPS 203/204/205 (National Institute of Standards and Technology, 2024) frente a la amenaza cuántica sobre la criptografía actual. Ambas trazan la hoja de ruta de BASTION (soportar passkeys y evaluar un cifrado híbrido post-cuántico), sin alterar el alcance del periodo.

8. PROTOTIPO DE INTERFAZ

Para validar el diseño se construyó un **prototipo funcional de alta fidelidad** que materializa las pantallas y flujos de los capítulos anteriores y permite interactuar con la lógica esencial (crear la bóveda, generar contraseñas, agregar credenciales, buscar y visualizar el archivo cifrado) antes de la implementación definitiva. Cumple el rol de maqueta navegable; el producto final se desarrollará en Python según el stack de la Tabla 10.

8.1 Pantalla principal de la bóveda

La Figura 5 muestra la pantalla principal: una bóveda con credenciales organizadas por categorías, buscador, indicadores de fortaleza y accesos directos para generar una contraseña o agregar una entrada. El diseño visual prioriza la claridad y la jerarquía de la información.

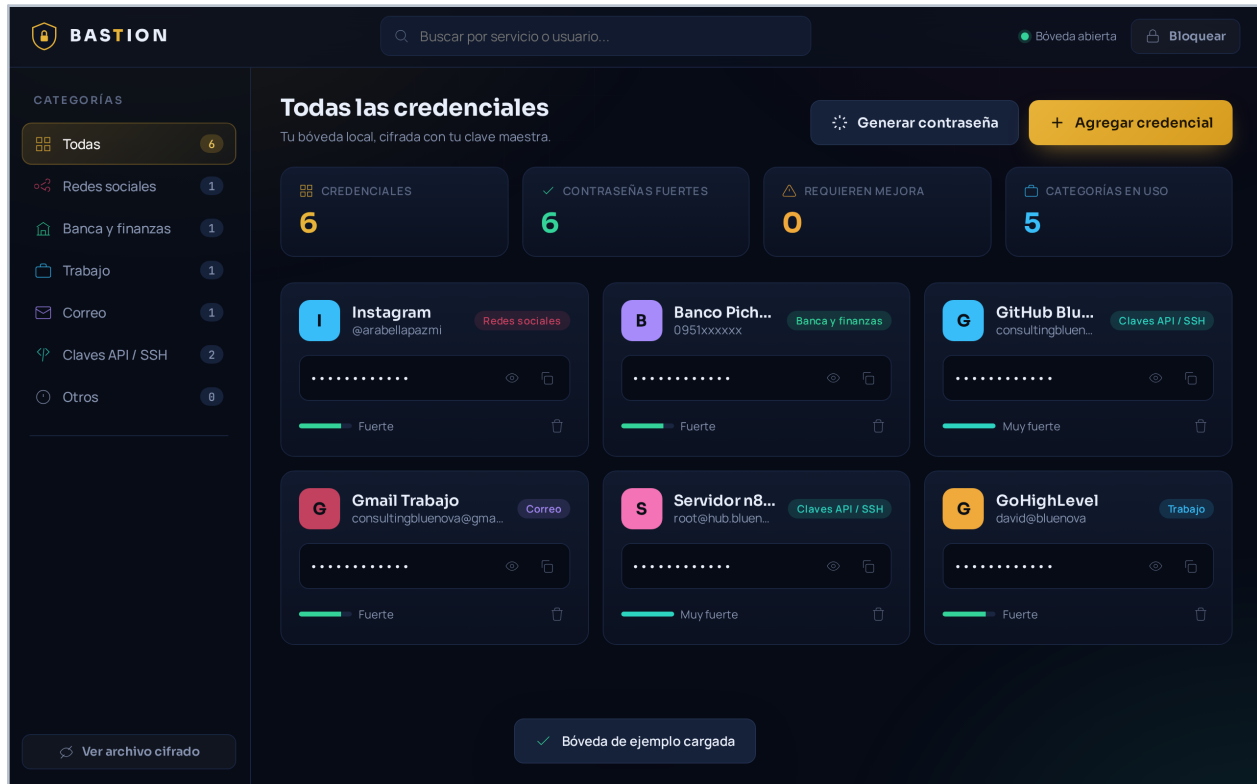


Figura 5. Prototipo de alta fidelidad: pantalla principal de la bóveda de BASTION.

8.2 Visualización del archivo cifrado

El prototipo incluye una vista que contrasta el contenido legible de la bóveda con su representación cifrada, ilustrando que, sin la contraseña maestra, los datos resultan ininteligibles. Esta pantalla refuerza la comprensión del modelo de seguridad del capítulo 7.

9. ALCANCE DEL PERIODO Y PROYECCIÓN

9.1 Alcance del periodo

El desarrollo de BASTION se concentra en junio de 2026, con el objetivo concreto de llegar a un MVP funcional al cierre del mes: generador de contraseñas, bóveda cifrada y una versión operativa del sistema. La Figura 6 sitúa ese alcance y, a su derecha, proyecta hasta dónde puede crecer el producto (categorías y búsqueda, doble factor con claves SSH y una eventual sincronización con auditoría LOPDP), sin que ello suponga una fecha de lanzamiento comercial. La proyección expresa el potencial de crecimiento, no un compromiso de entrega.

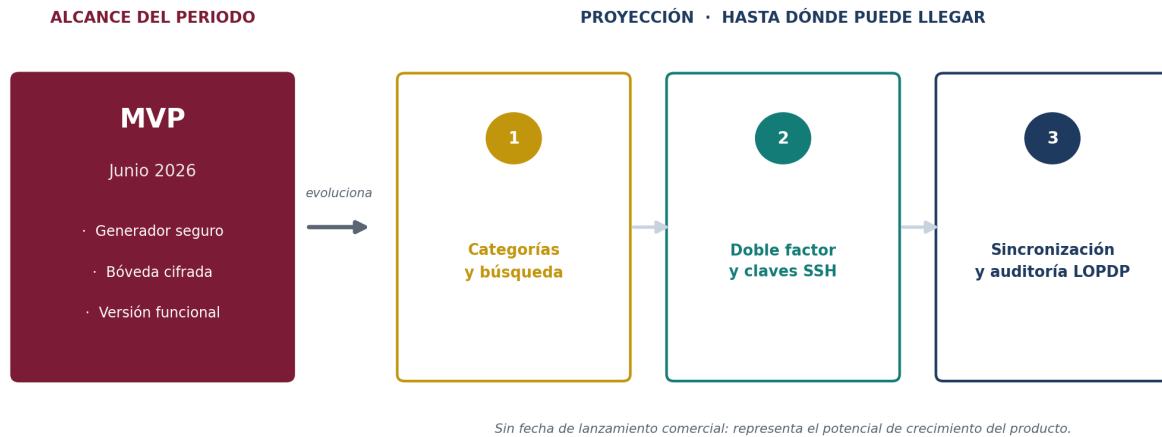


Figura 6. Alcance del periodo (MVP en junio) y proyección de BASTION.

9.2 Aplicación de los módulos de la asignatura

El diseño integra los contenidos de la asignatura. Del **Módulo 1** (fundamentos y lógica) se aplican la descomposición del problema y el diseño del flujo de control. Del **Módulo 2** (variables y tipos) se aplica la clasificación de las variables en str, int y bool (Tabla 2). Del **Módulo 3** (estructuras de control y funciones) se anticipan las decisiones, los ciclos y la modularización reflejada en la arquitectura en capas.

CONCLUSIONES

- El diseño previo a la codificación permitió comprender el problema de la gestión de contraseñas en profundidad y modelar una solución coherente antes de escribir una sola línea de código, lo que reduce el riesgo de retrabajo durante la implementación.
- Los diagramas de funcionalidad (casos de uso y flujo) y el diagrama de arquitectura en capas resultaron herramientas complementarias: los primeros describen qué hace el sistema y cómo interactúa el usuario, mientras que el segundo describe cómo se organiza internamente, cumpliendo así con los formatos exigidos por la actividad.
- El análisis comparativo con gestores del mercado confirmó que un modelo de seguridad sólido se apoya en estándares probados (AES-256-GCM, Argon2id, RFC 4086) y permitió posicionar a BASTION en un nicho técnico, local y auditable, sin pretender replicar la totalidad de las funciones comerciales.
- La construcción del prototipo de alta fidelidad validó las decisiones de diseño y aportó una guía visual concreta para la fase de implementación en Python, evidenciando la transferencia de los conceptos de la asignatura a un caso de negocio real de Blue Nova Consulting.
- El alcance delimitado del MVP, sumado a un cronograma realista de cinco sprints, hace de BASTION un proyecto académicamente abordable y profesionalmente útil, capaz de evolucionar de forma ordenada según el roadmap propuesto.

RECOMENDACIONES

- Implementar el MVP siguiendo estrictamente la arquitectura en capas diseñada, de modo que la lógica de negocio permanezca independiente de la interfaz y de la persistencia, facilitando futuras evoluciones como la migración de CLI a interfaz gráfica.
- No improvisar mecanismos criptográficos propios: emplear bibliotecas maduras y auditadas para AES-256-GCM y Argon2id, conforme a las guías del NIST citadas en este documento.
- Incorporar pruebas desde el sprint inicial, en especial para el generador y el motor de cifrado, por ser los componentes críticos de seguridad del sistema.
- Documentar el proyecto y versionarlo con control de cambios desde el primer commit, práctica indispensable para la mantenibilidad y para el trabajo en equipo dentro de Blue Nova.
- Al planificar las versiones 2.0 y 3.0, considerar tempranamente los requisitos de la LOPDP ecuatoriana en materia de protección de datos personales, anticipando la auditoría de seguridad.
- Mantener el diseño alineado con las tendencias de 2026: incorporar passkeys (FIDO2/WebAuthn) como método de autenticación en la hoja de ruta y evaluar un cifrado híbrido post-cuántico (por ejemplo, ML-KEM combinado con AES) para proteger la bóveda frente a la estrategia «recolectar ahora, descifrar después».

REFERENCIAS BIBLIOGRÁFICAS

Las referencias se presentan ordenadas alfabéticamente conforme a las normas APA en su séptima edición (American Psychological Association, 2020).

- American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.^a ed.). <https://doi.org/10.1037/0000165-000>
- Barker, E. (2020). *Recommendation for key management: Part 1, General* (NIST Special Publication 800-57 Part 1 Rev. 5). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-57pt1r5>
- Bass, L., Clements, P., & Kazman, R. (2012). *Software architecture in practice* (3.^a ed.). Addison-Wesley.
- Biryukov, A., Dinu, D., Khovratovich, D., & Josefsson, S. (2021). *Argon2 memory-hard function for password hashing and proof-of-work applications* (RFC 9106). Internet Engineering Task Force. <https://www.rfc-editor.org/info/rfc9106>
- Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language user guide* (2.^a ed.). Addison-Wesley.
- Dworkin, M. (2007). *Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC* (NIST Special Publication 800-38D). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-38D>
- Eastlake, D., Schiller, J., & Crocker, S. (2005). *Randomness requirements for security* (RFC 4086). Internet Engineering Task Force. <https://www.rfc-editor.org/info/rfc4086>
- FIDO Alliance. (2025). *Passkeys: Passwordless authentication*. <https://fidoalliance.org/passkeys/>
- Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.
- Grassi, P. A., Garcia, M. E., & Fenton, J. L. (2017). *Digital identity guidelines: Authentication and lifecycle management* (NIST Special Publication 800-63B). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-63b>
- M'Raihi, D., Machani, S., Pei, M., & Rydell, J. (2011). *TOTP: Time-based one-time password algorithm* (RFC 6238). Internet Engineering Task Force. <https://www.rfc-editor.org/info/rfc6238>
- National Institute of Standards and Technology. (2024). *Module-lattice-based key-encapsulation mechanism standard* (FIPS 203). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.FIPS.203>
- National Institute of Standards and Technology. (2025). *Digital identity guidelines* (NIST Special Publication 800-63-4). <https://doi.org/10.6028/NIST.SP.800-63-4>

Object Management Group. (2017). *Unified Modeling Language (UML), version 2.5.1*. OMG.
<https://www.omg.org/spec/UML/2.5.1>

OWASP Foundation. (2025). *Password storage cheat sheet*. OWASP Cheat Sheet Series.
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Pressman, R. S., & Maxim, B. R. (2021). *Software engineering: A practitioner's approach* (9.^a ed.). McGraw-Hill.

Sommerville, I. (2016). *Software engineering* (10.^a ed.). Pearson.

Stallings, W. (2023). *Cryptography and network security: Principles and practice* (8.^a ed.). Pearson.